ADITYA UNIVERSITY

Department of Artificial Intelligence and Machine Learning

B.Tech VII Semester

2026-2027

PROMPT ENGINEERING LAB

(231CS7S01)

ADITYA UNIVERSITY

Surampalem, Kakinada District

Andhra Pradesh-533437

UNIT-1

EXPERIMENT-1

Aim: To write a basic Python program that sends a prompt to the Gemini API using the Google GenAI client library and prints the model response "Hello, world!".

PROCEDURE

1. Install the Google GenAI Python library using pip.
2. Import the required modules (os and google.genai).
3. Set the GEMINI_API_KEY environment variable.
4. Create a GenAI client using the API key.
5. Send the prompt 'Hello, world!' to the gemini-2.5-flash model.
6. Receive the generated response.
7. Print the response on the screen.

PROGRAM

```
# -*- coding: utf-8 -*-
"""Untitled17.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1UtO8RScL7kJYQoUOrt-oDlgX5HvpL4ls
"""

!pip install -q google-genai

import os

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

from google import genai

client = genai.Client()

from google import genai

client = genai.Client(api_key="AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA")

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Hello, world!")
```

```
print(response.text)
```

OUTPUT

```
Hello, world!
```

(Generated response from the Gemini 2.5 Flash model)

RESULT

Thus, the basic Python program for sending a prompt to the Gemini API using the Google GenAI client library has been implemented successfully and the output has been verified.

EXPERIMENT-2

AIM:

To compare a baseline prompt with an enhanced prompt using the Gemini API and observe how prompt engineering improves the quality of AI-generated responses.

PROCEDURE

8. Install the Google GenAI library.
9. Import required modules and set the Gemini API key.
10. Create a GenAI client.
11. Define a baseline prompt and generate a response.
12. Define an enhanced prompt with detailed instructions.
13. Generate the enhanced response.
14. Compare both outputs and observe the improvements.

PROGRAM

Original file is located at
https://colab.research.google.com/drive/1-eKu1EGPSrtS48WFvATirXljz4JuvAF8

```
!pip install -U google-genai

from google import genai

import os

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"]
)

baseline_prompt = """
Write a one-paragraph bio of Ada Lovelace.
"""

# Generate baseline response
baseline_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=baseline_prompt
)

print(baseline_response.text)
```

```
enhanced_prompt = """
You are a professional historian.

Write a one-paragraph biography of Ada Lovelace.

Requirements:
1. Length: 120–150 words.
2. Mention her contributions to Charles Babbage's Analytical Engine.
3. Explain why she is considered the world's first computer programmer.
4. Use clear and formal language suitable for undergraduate students.
5. End with one sentence summarizing her historical significance.
"""

# Generate enhanced response
enhanced_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=enhanced_prompt
)

print("=" * 80)
print("BASELINE PROMPT")
print("=" * 80)
print(baseline_prompt)

print("\nBASELINE OUTPUT")
print("-" * 80)
print(baseline_response.text)

print("\n\n")

print("=" * 80)
print("ENHANCED PROMPT")
print("=" * 80)
print(enhanced_prompt)

print("\nENHANCED OUTPUT")
print("-" * 80)
print(enhanced_response.text)

print("\n" + "=" * 80)
print("COMPARISON")
print("=" * 80)

print("""
Baseline Prompt:
```

• Simple instruction
• Short response
• Limited details
• No formatting guidance

Enhanced Prompt:
• Includes role framing (Historian)
• Specifies length (120–150 words)
• Requests important facts
• Defines audience
• Gives formatting instructions
• Produces a more relevant, complete, and professional response
""")

OUTPUT

Baseline Output:
A short biography of Ada Lovelace.

Enhanced Output:
A detailed, well-structured biography (120–150 words) including her work on the Analytical Engine, why she is regarded as the first computer programmer, and her historical significance.

Comparison:
The enhanced prompt produces a more accurate, informative, and professional response than the baseline prompt.

RESULT

Thus, the program comparing a baseline prompt and an enhanced prompt using the Gemini API was implemented successfully, and the effect of prompt engineering on response quality was verified.

EXPERIMENT-3

AIM: To demonstrate iterative prompt refinement by comparing multiple prompts and observing improvements in the generated responses using the Gemini API.

PROCEDURE

1. Install the Google GenAI library using pip.
2. Import the required modules and set the Gemini API key.
3. Create a GenAI client.
4. Define three prompts with increasing levels of detail.
5. Generate responses for each prompt using the gemini-2.5-flash model.
6. Display all responses and compare the results.
7. Observe how prompt refinement improves the output.

PROGRAM

```
Original file is located at
    https://colab.research.google.com/drive/1dUXRKex3TIoPjqHuSb8UcKV9LgFm5tC4
"""

!pip install -U google-genai

from google import genai

import os

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"]
)

prompt1 = """
Summarize the plot of the Shakespearean play Romeo and Juliet in two sentences.
"""

response1 = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt1
)

prompt2 = """
```

```
Summarize the plot of the Shakespearean play Romeo and Juliet in exactly two sentences.
Use clear, simple, and formal language suitable for high school students.
"""

response2 = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt2
)

prompt3 = """
You are an English literature teacher.

Summarize the plot of Romeo and Juliet in exactly two sentences.

Requirements:
1. Mention that the story is set in Verona, Italy.
2. Include the conflict between the Montague and Capulet families.
3. Mention the tragic ending.
4. Highlight the central theme of love and fate.
5. Use clear and formal language suitable for undergraduate students.
"""

response3 = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt3
)

print("=" * 80)
print("ROUND 1: MINIMAL INSTRUCTION")
print("=" * 80)
print("Prompt:")
print(prompt1)

print("\nResponse:")
print(response1.text)

print("\n" + "=" * 80)
print("ROUND 2: LENGTH AND STYLE CONSTRAINTS")
print("=" * 80)
print("Prompt:")
print(prompt2)

print("\nResponse:")
print(response2.text)

print("\n" + "=" * 80)
```

```
print("ROUND 3: SPECIFIC CONTENT REQUIREMENTS")
print("=" * 80)
print("Prompt:")
print(prompt3)

print("\nResponse:")
print(response3.text)

print("\n" + "=" * 80)
print("COMPARISON OF THREE ROUNDS")
print("=" * 80)

print("""
```

Round 1:
- Minimal instruction.
- General two-sentence summary.
- No control over style or details.

Round 2:
- Adds length and style constraints.
- Produces a clearer and more readable summary.

Round 3:
- Specifies role, setting, conflict, theme, and ending.
- Produces the most complete, accurate, and well-structured summary.

```
""")
```

OUTPUT

The program displays three responses generated using progressively refined prompts for Romeo and Juliet, followed by a comparison showing that more detailed prompts produce clearer, more accurate, and better-structured outputs.

RESULT

Thus, the program demonstrating iterative prompt refinement using the Gemini API has been implemented successfully and the output has been verified.

EXPERIMENT-4

AIM

To diagnose prompt failures caused by ambiguity and contradictions, refine the prompt, and compare the outputs using the Google GenAI client library.

PROCEDURE

Install the Google GenAI library using pip.

Import the required modules.

Set the GEMINI_API_KEY.

Create the GenAI client.

Create a vague/contradictory prompt.

Generate the model response.

Refine the prompt with clear instructions.

Generate the refined response.

Compare both outputs and analyze the improvement.

Display the conclusion.

PROGRAM

Original file is located at
https://colab.research.google.com/drive/1q7j9S8nwHuvq8HMc-FbIon1xTCtKX3WC

"""

!pip install -U google-genai

from google import genai

import os

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(
  api_key=os.environ["GEMINI_API_KEY"])

vague_prompt = """
Write a detailed summary of Artificial Intelligence in one sentence.
Make it both very technical and easy for a 10-year-old to understand.

```
"""

vague_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=vague_prompt)

refined_prompt = """
You are an AI teacher.

Write exactly one sentence (30–40 words) explaining Artificial Intelligence.

**Requirements:**
1. Use simple English suitable for a 10-year-old student.
2. Mention that AI enables computers to learn and solve problems.
3. Avoid technical jargon.
4. Follow this style example:
   "Plants use sunlight to make food, helping them grow and stay healthy."
"""

refined_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=refined_prompt
)

print("=" * 80)
print("STEP 1: VAGUE / CONTRADICTORY PROMPT")
print("=" * 80)

print("Prompt:")
print(vague_prompt)

print("\nModel Output:")
print(vague_response.text)

print("\nFailure Analysis:")
print("- Ambiguity: The prompt requests a detailed summary in one sentence.")
print("- Contradiction: It asks for both technical language and language suitable for a 10-year-old.")
print("- Missing Format Guidance: No word limit or writing style is specified.")

print("\n" + "=" * 80)
print("STEP 2: REFINED PROMPT")
print("=" * 80)

print("Prompt:")
print(refined_prompt)
```

```
print("\nModel Output:")
print(refined_response.text)

print("\nImprovement Analysis:")
print("- Clear audience specified.")
print("- Exact sentence length provided.")
print("- Technical jargon is prohibited.")
print("- Key concepts are explicitly mentioned.")
print("- An example guides the writing style.")

print("\n" + "=" * 80)
print("CONCLUSION")
print("=" * 80)

print("The refined prompt produces a clearer, more accurate, and consistent response "
    "because it removes ambiguity, resolves contradictions, and provides explicit "
    "instructions with an example.")
```

OUTPUT

STEP 1: Vague Prompt
The model generates an inconsistent response due to ambiguity and contradictory instructions.

STEP 2: Refined Prompt
The model generates a clear, simple, and consistent response that satisfies all requirements.

CONCLUSION
The refined prompt produces a more accurate and reliable output.

RESULT

Thus, the Python program to diagnose prompt failures, refine the prompt, and compare the outputs using the Gemini API was implemented successfully and the results were verified.

UNIT-2

EXPERIMENT-1

AIM

To perform Zero-Shot and Few-Shot prompting for sentiment classification using the Google GenAI client library and compare the outputs.

PROCEDURE

Install Google GenAI library.

Import required modules.

Set the GEMINI_API_KEY.

Create the GenAI client.

Define the input sentence.

Create a Zero-Shot prompt.

Create a Few-Shot prompt with examples.

Generate responses using Gemini 2.5 Flash.

Compare both outputs.

Display the conclusion.

PROGRAM

Original file is located at
https://colab.research.google.com/drive/1C0728hUxUMD6MNpUPir8AxYabMZbt032

```
============================================================
Experiment-2.1: Few-Shot vs. Zero-Shot Prompting
Task: Sentiment Classification
Model: Gemini 2.5 Flash
============================================================
"""

!pip install -U google-genai

from google import genai

import os

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
7gs0UlOjgx2zUVxcD-liLLHA"
```

```
client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"]
)

input_text = "The movie had excellent acting, but the story was slow."

zero_shot_prompt = f"""
Classify the sentiment of the following sentence as
Positive, Negative, or Neutral.

Sentence:
"{input_text}"
"""

zero_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=zero_shot_prompt
)

few_shot_prompt = f"""
Classify the sentiment as Positive, Negative, or Neutral.

Example 1:
Sentence: I absolutely loved the food.
Sentiment: Positive

Example 2:
Sentence: The service was terrible.
Sentiment: Negative

Example 3:
Sentence: The meeting was held at 10 AM.
Sentiment: Neutral

Now classify the following sentence.

Sentence: "{input_text}"
Sentiment:
"""

few_response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=few_shot_prompt
)
```

```
print("=" * 80)
print("ZERO-SHOT PROMPT")
print("=" * 80)

print(zero_shot_prompt)

print("\nModel Output:")
print(zero_response.text)

print("\n" + "=" * 80)
print("FEW-SHOT PROMPT")
print("=" * 80)

print(few_shot_prompt)

print("\nModel Output:")
print(few_response.text)

print("\n" + "=" * 80)
print("COMPARISON")
print("=" * 80)


Zero-Shot Prompt
----------------
• No examples are provided.
• Model relies only on the instruction.
• Output may vary in wording and explanation.

Few-Shot Prompt
---------------
• Includes three input-output examples.
• Guides the model toward the expected format.
• Produces more consistent and reliable results.
• Better follows the required output style.

Conclusion
----------
Few-shot prompting generally improves:
1. Accuracy
2. Consistency
3. Adherence to the provided examples
""")
```

OUTPUT

ZERO-SHOT PROMPT
Sentiment: Positive

FEW-SHOT PROMPT
Sentiment: Positive

COMPARISON
• Zero-shot uses only instructions.
• Few-shot uses example input-output pairs.
• Few-shot provides more consistent and accurate results.

RESULT

Thus, the Python program for comparing Zero-Shot and Few-Shot prompting using the Gemini API was implemented successfully and the outputs were verified.

EXPERIMENT-2

AIM: To perform Role-Based and Negative Prompting using the Google GenAI client library and evaluate their influence on the generated responses.

PROCEDURE

15. Install the Google GenAI library.
16. Import the required modules.
17. Set the GEMINI_API_KEY.
18. Create the GenAI client.
19. Create a role-based prompt.
20. Create a negative prompt.
21. Generate responses using Gemini 2.5 Flash.
22. Display both responses.
23. Compare and evaluate the outputs.
24. Draw the conclusion.

PROGRAM

Original file is located at

https://colab.research.google.com/drive/1oJLGA_R4CWSlEwMlPkPVEwACMRKuxPsW#scrollTo=6577bcda

```
# Experiment 2.2: Role-Based & Negative Prompting
"""

!pip install -U google-genai

from google import genai
import os
os.environ["GEMINI_API_KEY"]="YOUR_GEMINI_API_KEY"
client=genai.Client(api_key=os.environ["GEMINI_API_KEY"])

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"]
)

role_prompt="""You are an experienced financial advisor.

Explain how a college student can save money.
Use simple English.
```

```
Give five practical tips.
"""

negative_prompt="""Explain how a college student can save money.

Use simple English.
Give five practical tips.
Do NOT mention any brand names.
Do NOT recommend any banking app.
Do NOT include advertisements.
"""

role_response=client.models.generate_content(model="gemini-2.5-flash",contents=role_prompt)
negative_response=client.models.generate_content(model="gemini-2.5-
flash",contents=negative_prompt)

print("ROLE-BASED PROMPT\n")
print(role_response.text)
print("\nNEGATIVE PROMPT\n")
print(negative_response.text)
print("\nEVALUATION")
print("- Role-based prompting gives professional advice.")
print("- Negative prompting suppresses unwanted content.")
print("- Combining both improves response quality.")
```

OUTPUT

ROLE-BASED PROMPT
The model provides professional financial advice with five practical tips.

NEGATIVE PROMPT
The model provides the same guidance without mentioning any brand names, banking apps, or advertisements.

EVALUATION
- Role-based prompting improves the style and expertise of the response.
- Negative prompting suppresses unwanted content.
- Combining both techniques produces more relevant and controlled outputs.

RESULT

Thus, the Python program for Role-Based and Negative Prompting using the Google GenAI client library was implemented successfully, and the influence of both prompting techniques was verified.

EXPERIMENT-3

Aim

To write a Python program that demonstrates Constraint Specification and Iterative Refinement using the Google Gemini API by generating summaries with progressively refined prompts.

Procedure

1. Open Google Colab and install the google-genai library.
2. Import the required modules and configure the Gemini API key.
3. Create a Gemini client object.
4. Define the input article as a multi-line string.
5. Create multiple prompts with increasing levels of constraints.
6. Send each prompt to the Gemini model using generate_content().
7. Display the generated outputs for each refinement cycle.
8. Compare the outputs to observe the effect of prompt refinement.

Program

Original file is located at
https://colab.research.google.com/drive/1-clyXhyPs2oP1HSVOo0yn2onfvDN1waT#scrollTo=aeeb46d7

```
# Constraint Specification & Iterative Refinement using Google Gemini
Run this notebook in Google Colab.
"""

!pip -q install -U google-genai

os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(
    api_key=os.environ["GEMINI_API_KEY"]
)

article = '''
Artificial Intelligence (AI) is transforming industries by enabling machines to perform tasks that traditionally require human intelligence. Machine Learning is a subset of AI that allows computers to learn from data without being explicitly programmed. Deep Learning uses neural networks with multiple layers. AI is used in healthcare, finance, education, cybersecurity, and robotics. Challenges include bias, privacy, transparency, and ethics.
'''

prompts = [
f"""Summarize the following article.\n\n{article}""",
```

```
f"""Summarize the article in 80-100 words using simple English.\n\n{article}""",
f"""Summarize the article with these constraints:
- Exactly 5 bullet points
- 80-100 words
- Mention Artificial Intelligence, Machine Learning, Deep Learning, Applications, Challenges
- Use simple English

{article}"""
]

titles=['Basic Prompt','Refinement Cycle 1','Refinement Cycle 2']

for t,p in zip(titles,prompts):
    print('='*60)
    print(t)
    print('='*60)
    resp=client.models.generate_content(model='gemini-2.5-flash',contents=p)
    print(resp.text)
    print()
```

Sample Output

```
============================================================
Basic Prompt
============================================================
Artificial Intelligence (AI) is transforming industries...
(Summary generated by Gemini)

============================================================
Refinement Cycle 1
============================================================
(Simple English summary of 80–100 words.)

============================================================
Refinement Cycle 2
============================================================
• Artificial Intelligence...
• Machine Learning...
• Deep Learning...
• Applications...
• Challenges...
```

Result

Thus, the Python program was executed successfully and demonstrated Constraint Specification and Iterative Refinement using the Google Gemini API. The refined prompts produced more structured and accurate summaries than the basic prompt.

UNIT-3

EXPERIMENT-1

Aim:

To demonstrate Structured Format Prompting using the Google Gemini API by instructing the model to generate responses in a specified structure such as bullet lists and Markdown tables.

Procedure:

1. Open Google Colab.
2. Install the Google GenAI library.
3. Import the required Python modules.
4. Set the Gemini API key.
5. Create a Gemini client.
6. Define a prompt requesting output in bullet-list and Markdown table formats.
7. Generate the response using the Gemini model.
8. Display the generated output.

Program:

Original file is located at
https://colab.research.google.com/drive/1Y_HNgFxYzjg9i0WGwAhgpXE-HfTgv195

```
# Structured Format Prompting using Google Gemini

Run this notebook in Google Colab.
"""

# Install Google GenAI library
!pip install -q -U google-genai

import os
from google import genai

# Set your Gemini API Key
os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
7gs0UlOjgx2zUVxcD-liLLHA"

# Create Gemini client
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])

# Prompt demonstrating Structured Format Prompting
prompt = """
List three benefits of daily exercise.
```

```
Instructions:
1. First, provide the answer as a bullet list.
2. Then, provide the same information as a Markdown table.
3. The table should contain the columns:
   - Benefit
   - Description
4. Use simple English.
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt)

print("=" * 60)
print("Structured Format Prompting")
print("=" * 60)
print(response.text)
```

Sample Output:

```
============================================================
Structured Format Prompting
============================================================

• Improves physical health by strengthening muscles and the heart.
• Increases energy levels and reduces tiredness.
• Improves mental health by reducing stress and improving mood.

| Benefit | Description |
| Physical Health | Strengthens muscles and improves heart health. |
| More Energy | Helps you stay active throughout the day. |
| Better Mental Health | Reduces stress and improves mood. |
```

Result:

Thus, the program was executed successfully and the Gemini model generated the response in the required structured format consisting of bullet lists and a Markdown table.

EXPERIMENT-2

AIM:

To generate a dataset in valid JSON and YAML formats using the Google Gemini API, validate the generated syntax using JSON and YAML parsers, and display the validation results.

PROCEDURE:

1. Install the required libraries (google-genai and PyYAML).
2. Import the required modules.
3. Set the Gemini API key.
4. Create a Gemini client.
5. Provide a prompt requesting a dataset in JSON and YAML formats.
6. Generate the response using the Gemini model.
7. Validate the JSON output using the json parser.
8. Validate the YAML output using the yaml parser.
9. Display the generated data and validation results.

PROGRAM:

```
Original file is located at
    https://colab.research.google.com/drive/144rbHb0jKlSfj0yoUd31canT5E2tT0yf

# JSON/YAML Generation and Validation using Google Gemini
Run in Google Colab.
"""

# Install required libraries
!pip install -q -U google-genai pyyaml

import os
import json
import yaml
from google import genai

# Set your Gemini API Key
os.environ["GEMINI_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
7gs0UlOjgx2zUVxcD-liLLHA"

client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
```

```
prompt = '''
Dataset Description:
Create a dataset of three books with the following fields:
- title
- author
- publication_year

Return:
1. Valid JSON only.
2. Valid YAML only.
Do not include explanations.
'''

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("Gemini Response:")
print(response.text)

# Split JSON and YAML (expects JSON first, YAML second)
parts = response.text.strip().split("\n\n",1)

if len(parts)==2:
    json_text, yaml_text = parts
else:
    json_text = response.text
    yaml_text = ""

print("\n--- JSON Validation ---")
try:
    obj=json.loads(json_text)
    print("Valid JSON")
except Exception as e:
    print("Invalid JSON:",e)

print("\n--- YAML Validation ---")
try:
    if yaml_text:
        yaml.safe_load(yaml_text)
        print("Valid YAML")
    else:
        print("YAML section not found.")
```

```
except Exception as e:
    print("Invalid YAML:",e)
```

SAMPLE OUTPUT:

Gemini Response:

JSON:

```
[
  {
    "title":"Book A",
    "author":"Author A",
    "publication_year":2022
  },
  {
    "title":"Book B",
    "author":"Author B",
    "publication_year":2021
  },
  {
    "title":"Book C",
    "author":"Author C",
    "publication_year":2020
  }
]
```

YAML:

```
- title: Book A
  author: Author A
  publication_year: 2022
- title: Book B
  author: Author B
  publication_year: 2021
- title: Book C
  author: Author C
  publication_year: 2020
```

JSON Validation: Valid JSON
YAML Validation: Valid YAML

RESULT:

Thus, the Python program for generating JSON and YAML data using the Google Gemini API and validating the generated syntax using JSON and YAML parsers was implemented successfully.

EXPERIMENT-3:

AIM:
To develop a Python program using the Google GenAI client library to compare Direct Answer Baseline, Zero-shot Chain-of-Thought prompting, and Task Decomposition for solving a multi-step logic puzzle.

PROCEDURE:

1. Install the Google GenAI Python library using pip.
2. Import the required modules.
3. Set the GEMINI_API_KEY environment variable.
4. Create a Gemini client using the API key.
5. Define a multi-step logic puzzle.
6. Generate a direct-answer baseline response.
7. Generate a Zero-shot Chain-of-Thought response.
8. Decompose the task into sub-questions and collect partial answers.
9. Combine the partial answers into the final solution.
10. Compare the three approaches and display the results.

PROGRAM:

Original file is located at
https://colab.research.google.com/drive/11QzT5rgnijiQodmim5CfVXvd3chOlyPV


```
# Chain-of-Thought & Task Decomposition using Google Gemini
Run this notebook in Google Colab.
"""

# Install library
!pip install -q -U google-genai

import os
from google import genai

# Set your API key
os.environ["GEMINI_API_KEY"]="AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-7gs0UlOjgx2zUVxcD-liLLHA"

client=genai.Client(api_key=os.environ["GEMINI_API_KEY"])

problem="""
A farmer needs to cross a river with a wolf, a goat, and a cabbage.
The boat can carry only the farmer and one item.
```

```
If left alone, the wolf eats the goat and the goat eats the cabbage.
Find a valid sequence of crossings.
"""

# Direct-answer baseline
baseline_prompt=f"""
Solve the following problem and give only the final answer.

{problem}
"""

baseline=client.models.generate_content(
    model="gemini-2.5-flash",
    contents=baseline_prompt
)

print("="*60)
print("DIRECT ANSWER BASELINE")
print("="*60)
print(baseline.text)

# Zero-shot Chain-of-Thought
cot_prompt=f"""
Let's think step by step.

{problem}

Explain your reasoning before giving the final answer.
"""

cot=client.models.generate_content(
    model="gemini-2.5-flash",
    contents=cot_prompt
)

print("\n"+"="*60)
print("ZERO-SHOT CHAIN-OF-THOUGHT")
print("="*60)
print(cot.text)

# Task decomposition
subquestions=[
"Identify the constraints of the puzzle.",
"What should be the first crossing?",
"What should be the remaining sequence of crossings?",
"Provide the final ordered solution."
```

```
]

answers=[]
for i,q in enumerate(subquestions,1):
    r=client.models.generate_content(
        model="gemini-2.5-flash",
        contents=f"{problem}\n\nQuestion {i}: {q}"
    )
    answers.append(r.text)
    print(f"\nSub-question {i}:")
    print(r.text)

combine_prompt="Combine the following partial answers into one correct final solution:\n\n"
for i,a in enumerate(answers,1):
    combine_prompt+=f"Answer {i}:\n{a}\n\n"

combined=client.models.generate_content(
    model="gemini-2.5-flash",
    contents=combine_prompt
)

print("\n"+"="*60)
print("COMBINED FINAL SOLUTION")
print("="*60)
print(combined.text)

print("\nComparison:")
print("- Baseline: Direct answer only")
print("- Zero-shot CoT: Includes reasoning and final answer")
print("- Task Decomposition: Solves using sequential sub-questions and combines results")
```

OUTPUT:

```
============================================================
DIRECT ANSWER BASELINE
============================================================
(Displays the direct solution.)

============================================================
ZERO-SHOT CHAIN-OF-THOUGHT
============================================================
(Displays reasoning followed by the final solution.)

============================================================
COMBINED FINAL SOLUTION
============================================================
(Displays the combined solution obtained from task decomposition.)
```

Comparison:
- Baseline: Direct answer only
- Zero-shot CoT: Includes reasoning and final answer
- Task Decomposition: Sequential sub-questions with combined solution.

RESULT:

Thus, the Python program for implementing Chain-of-Thought prompting and Task Decomposition using the Google GenAI client library was executed successfully, and the responses from the different prompting strategies were compared successfully.

UNIT-4

EXPERIMENT-1

AIM

To build a simple LCEL (LangChain Expression Language) chain that accepts a fixed instruction, sends it to the Gemini language model, prints the generated summary, and verifies successful end-to-end execution.

PROCEDURE

1. Install the required LangChain and Google Gemini libraries.
2. Import the required Python modules.
3. Set the Google API key.
4. Initialize the Gemini language model.
5. Create a prompt template for text summarization.
6. Build an LCEL chain using the prompt, model, and output parser.
7. Provide input text to the chain.
8. Execute the chain and display the generated summary.
9. Verify successful end-to-end execution.

PROGRAM

Original file is located at
https://colab.research.google.com/drive/1keXum1NidX4T3KxclXSjhbeiecgywKsw

```
# Building a Simple LCEL Chain

Run this notebook in Google Colab.
"""

# Install required libraries
!pip install -q -U langchain langchain-google-genai

import os
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI

# Set your Gemini API key
os.environ["GOOGLE_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
7gs0UlOjgx2zUVxcD-liLLHA"

# Initialize the Gemini model
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0)
```

```
# Create a prompt template
prompt = ChatPromptTemplate.from_template(
    "Summarize this text in 3-4 sentences:\n\n{text}")

# Build a simple LCEL chain
chain = prompt | llm | StrOutputParser()

# Input text
text = '''
Artificial Intelligence (AI) enables computers to perform tasks that normally require human intelligence.
Machine Learning is a subset of AI that learns from data.
Deep Learning uses neural networks with many layers.
AI is widely used in healthcare, education, finance, and cybersecurity.
'''

# Run the chain
result = chain.invoke({"text": text})

print("="*60)
print("LCEL CHAIN OUTPUT")
print("="*60)
print(result)

print("\nEnd-to-end execution completed successfully.")
```


OUTPUT

```
============================================================
LCEL CHAIN OUTPUT
============================================================
Artificial Intelligence enables computers to perform intelligent tasks. Machine Learning allows systems to learn from data, while Deep Learning uses multi-layer neural networks. AI has applications in healthcare, education, finance, and cybersecurity.

End-to-end execution completed successfully.
```

RESULT

Thus, a simple LCEL chain was successfully created using LangChain and the Gemini model. The chain accepted a fixed instruction, generated the required summary, and executed successfully from input to output.

EXPERIMENT-2

AIM:
To implement basic data indexing for Retrieval-Augmented Generation (RAG) by loading sample documents, splitting them into chunks, generating embeddings, storing them in an in-memory vector store, and verifying retrieval using similarity search.

PROCEDURE:
1. Install the required Python libraries.
2. Import the required modules.
3. Set the Google API key.
4. Create sample documents.
5. Split the documents into uniform chunks.
6. Generate embeddings for each chunk.
7. Store the embeddings in an in-memory FAISS vector store.
8. Perform similarity search using a sample query.
9. Display the retrieved chunks and verify the indexing process.

PROGRAM:
Original file is located at
https://colab.research.google.com/drive/1XS7VVI7Tq238MEsI3P1GVAiIG6aKadlp

```
# Basic Data Indexing for RAG using LangChain and Gemini

Run this notebook in Google Colab.
"""
# Install required libraries
!pip install -q -U \
google-genai \
langchain \
langchain-core \
langchain-community \
langchain-google-genai \
langchain-text-splitters \
faiss-cpu
!pip install -q -U langchain-huggingface sentence-transformers
from google import genai
import os
client = genai.Client(api_key=os.environ["GOOGLE_API_KEY"])

#for model in client.models.list():
  # print(model.name)
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS
# Set your Gemini API key
os.environ["GOOGLE_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
```

```
7gs0UlOjgx2zUVxcD-liLLHA"

# Sample documents
documents = [
    Document(page_content="Artificial Intelligence enables machines to perform intelligent tasks."),
    Document(page_content="Machine Learning is a subset of Artificial Intelligence that learns from
data."),
    Document(page_content="Deep Learning uses multi-layer neural networks for complex learning
tasks."),
    Document(page_content="Generative AI creates text, images, code, and other content from
prompts.")
]

# Split documents into uniform chunks
splitter = RecursiveCharacterTextSplitter(
    chunk_size=200,
    chunk_overlap=0
)

chunks = splitter.split_documents(documents)

print("Number of Chunks:", len(chunks))
for i, chunk in enumerate(chunks, start=1):
    print(f'Chunk {i}: {chunk.page_content}')

# Create embedding model
from langchain_huggingface import HuggingFaceEmbeddings

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
# Store embeddings in an in-memory FAISS vector store
vector_store = FAISS.from_documents(chunks, embeddings)

print("\nEmbeddings generated successfully.")
print("Number of vectors stored:", vector_store.index.ntotal)

# Inspect similarity search
query = "Explain Machine Learning"

results = vector_store.similarity_search(query, k=2)

print("\nTop Matching Chunks:")
for i, doc in enumerate(results, start=1):
    print(f'Result {i}:')
    print(doc.page_content)
```

```
    print("-"*50)

print("\nEnd-to-end indexing completed successfully.")
```

OUTPUT:

```
Number of Chunks: 4
Embeddings generated successfully.
Number of vectors stored: 4

Top Matching Chunks:
Result 1:
Machine Learning is a subset of Artificial Intelligence that learns from data.
Result 2:
Artificial Intelligence enables machines to perform intelligent tasks.
End-to-end indexing completed successfully.
```

RESULT:

Thus, the basic data indexing process for Retrieval-Augmented Generation (RAG) was implemented successfully by loading documents, splitting them into chunks, generating embeddings, storing them in an in-memory vector store, and retrieving relevant documents using similarity search.

EXPERIMENT-3

AIM:

To construct and execute a basic Retrieval-Augmented Generation (RAG) chain that receives a user query, retrieves the top-k relevant document chunks, combines the retrieved context with the query, sends the prompt to a Large Language Model (LLM), and compares the response with a prompt executed without retrieval.

PROCEDURE:

1. Install the required Python libraries.
2. Import the required LangChain and Google Gemini modules.
3. Set the Google API key.
4. Create sample documents for indexing.
5. Split the documents into chunks.
6. Generate embeddings and create a FAISS vector store.
7. Receive a user query and retrieve the top-k relevant chunks.
8. Construct a prompt using the retrieved context and the query.
9. Send the prompt to the Gemini model and generate the answer.
10. Compare the retrieved answer with the answer generated without retrieval.

PROGRAM:

Original file is located at
https://colab.research.google.com/drive/1MgNepRdfTnURSfYwkuzY-78QcR5pV6DY


```
# Constructing & Running a Basic RAG Chain using LangChain and Gemini

Run this notebook in Google Colab.
"""

# Install required libraries
!pip install -q -U langchain langchain-core langchain-community \
langchain-google-genai langchain-text-splitters \
langchain-huggingface sentence-transformers faiss-cpu
import os
from langchain_core.documents import Document
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_google_genai import ChatGoogleGenerativeAI

# Set your Gemini API key
os.environ["GOOGLE_API_KEY"] = "AQ.Ab8RN6L_xc_KE0RGScF6KQXgi-
```

```
7gs0UlOjgx2zUVxcD-liLLHA"

# Sample documents
documents = [
    Document(page_content="Artificial Intelligence enables machines to perform tasks requiring
human intelligence."),
    Document(page_content="Machine Learning is a subset of AI that learns patterns from data."),
    Document(page_content="Deep Learning uses multi-layer neural networks."),
    Document(page_content="Generative AI creates text, images, code, and other content.")
]

# Split documents
splitter = RecursiveCharacterTextSplitter(chunk_size=200, chunk_overlap=0)
chunks = splitter.split_documents(documents)

# Create embeddings and vector store
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)
vector_store = FAISS.from_documents(chunks, embeddings)

# Initialize Gemini LLM
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    temperature=0
)

def rag_chain(query, k=2):
    print("="*60)
    print("User Query:")
    print(query)

    docs = vector_store.similarity_search(query, k=k)

    print("\nRetrieved Context:")
    context = ""
    for i, d in enumerate(docs, 1):
        print(f"{i}. {d.page_content}")
        context += d.page_content + "\n"

    prompt = f"""
Use the following context to answer the question.

Context:
{context}
```

```
Question:
{query}

Answer:
"""

    response = llm.invoke(prompt)

    print("\nAnswer with Retrieval:")
    print(response.content)

    baseline = llm.invoke(query)

    print("\nAnswer without Retrieval:")
    print(baseline.content)

    print("\nComparison:")
    print("- Retrieval uses relevant context from the vector store.")
    print("- Baseline relies only on the model's knowledge.")

# Sample queries
rag_chain("What is Machine Learning?")
print("\n")
rag_chain("What is Generative AI?")
```

OUTPUT:

```
User Query: What is Machine Learning?

Retrieved Context:
1. Machine Learning is a subset of AI that learns patterns from data.
2. Artificial Intelligence enables machines to perform tasks requiring human intelligence.

Answer with Retrieval:
Machine Learning is a branch of AI that learns patterns from data.

Answer without Retrieval:
Machine Learning is a field of AI that enables systems to learn from data.

Comparison:
• Retrieval uses relevant context from the vector store.
• Baseline relies only on the model's knowledge.
```

RESULT:

Thus, the basic RAG chain was implemented successfully. The system retrieved relevant document chunks, combined them with the user query, generated context-aware responses using the LLM, and demonstrated improved factual grounding compared with a prompt executed without retrieval.